试菜搭配（pairing）

本题正解所涉及的知识点主要包括**二分答案、滑动窗口（前缀和）、贪心/阈值图结论与组合计数**。其中“二分答案”属于“基础算法”，大纲标注学习难度系数为 3；“滑动窗口/前缀和”属于“算法策略”，大纲标注学习难度系数为 3；“贪心结论”属于“算法策略”，大纲标注学习难度系数为 3。本题所涉知识点均在 NOI 大纲所规定的 CSP-S（提高级）难度系数范围内。

题目关键结论

把编号为 $1..n$ 的食材柜看作一条链，第 i 个柜有 a_i 份食材。给定 x，能配对的柜子必须满足编号差 $|i-j| \geq x$。设 $S = \sum a_i$，则**最多能做出的菜数**为

$$\text{dishes}(x) = \min\left(\lfloor S/2 \rfloor,\ S - \max_{\text{长度 } x \text{ 的连续窗口}} \text{窗口和}\right)$$

解释：任何**长度为 x 的连续窗口**内任意两个柜子编号差都 $< x$，互相不能配对，故窗口内的食材必须和窗口外的食材配对，最多配 $S -$ 窗口和 道；此外总数上限为 $\lfloor S/2 \rfloor$。对所有窗口取 $\min$ 即得（这是“链上距离受限最大匹配”的经典结论）。

“能做出至少 y 道菜”关于 x 单调：x 越大，窗口越宽，窗口和越大，$\text{dishes}(x)$ 越小。因此可以对 x 二分。

测试点 1~2：$n \leq 10,\ y \leq 10$

思路

枚举 $x = 1..n$，对每个 x 用回溯求出最大匹配，判断是否 $\geq y$。$n \leq 10$ 时指数级回溯完全可行。

复杂度

枚举 x 为 $O(n)$，每个 x 回溯为指数级；$n \leq 10$ 可通过。

```cpp
#include <bits/stdc++.h>
using namespace std;
int n, a[12], rem[12];
long long y;
int dfs(int x) {
    int p = -1;
    for (int i = 0; i < n; ++i) if (rem[i] > 0) { p = i; break; }
    if (p == -1) return 0;
    rem[p]--;
    int best = dfs(x);
    for (int j = p + x; j < n; ++j)
        if (rem[j] > 0) { rem[j]--; best = max(best, 1 + dfs(x)); rem[j]++; }
    rem[p]++;
    return best;
}
int main() {
    cin >> n >> y;
```

```
    for (int i = 0; i < n; ++i) cin >> a[i];
    int ans = -1;
    for (int x = 1; x <= n; ++x) {
        for (int i = 0; i < n; ++i) rem[i] = a[i];
        if (dfs(x) >= y) ans = x;
    }
    cout << ans << '\n';
    return 0;
}
```

测试点 3~7：$n \le 2000$

思路

直接用上面的结论。枚举 x 后，用 $O(n)$ 或 $O(n^2)$ 求长度为 x 的窗口和最大值，套公式 $\min(S/2,\ S - \max\text{窗口和})$ 判断是否 $\ge y$。

复杂度

枚举 x 为 $O(n)$，每个 x 枚举窗口 $O(n^2)$，总 $O(n^3)$；$n \le 2000$ 时用滑动窗口可降至 $O(n^2)$。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
int n; ll y, a[2005];
ll dishes(int x) {
    ll S = 0; for (int i = 0; i < n; ++i) S += a[i];
    ll cur = 0, mx = 0;
    for (int i = 0; i < n; ++i) {
        cur += a[i];
        if (i >= x) cur -= a[i - x];
        mx = max(mx, cur);
    }
    return min(S / 2, S - mx);
}
int main() {
    cin >> n >> y;
    for (int i = 0; i < n; ++i) cin >> a[i];
    int ans = -1;
    for (int x = 1; x <= n; ++x) if (dishes(x) >= y) ans = x;
    cout << ans << '\n';
    return 0;
}
```

测试点 8~10：$a_i = 1$（所有食材柜均只有 1 份）

思路

此时 $S=n$，长度为 x 的窗口和恒为 x，于是 $\text{dishes}(x)=\min(\lfloor n/2\rfloor,\ n-x)$。直接求最大的 x 使 $\min(\lfloor n/2\rfloor,\ n-x)\ge y$，即 $x\le n-y$ 且 $\lfloor n/2\rfloor\ge y$。答案即 $n-y$（不满足则 -1）。

复杂度

$O(1)$。

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
int main() {
    int n; ll y; cin >> n >> y;
    for (int i = 0; i < n; ++i) { int t; cin >> t; }
    if ((ll)n / 2 < y) { cout << -1 << '\n'; return 0; }
    cout << (ll)n - y << '\n';
    return 0;
}
```

测试点 11~20：$n\le 10^5,\ y\le 10^{12}$

思路

“能做出 $\ge y$ 道菜”关于 x 单调，对 x 二分；每个 x 用滑动窗口 $O(n)$ 求最大窗口和。注意 S,y 可达 10^{12}（$a_i\le 10^9$，$n\le 10^5$），全程使用 `long long`。本题的关键陷阱：**不要漏掉 $\lfloor S/2\rfloor$ 上限**——当 $y>\lfloor S/2\rfloor$ 时应直接输出 -1，且 `dishes(x)` 必须取 $\min(S/2,\ S-\max窗口和)$。

复杂度

$O(n\log n)$，空间 $O(n)$。

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N = 100010;
int n; ll y, a[N];
ll dishes(ll x) {
    ll S = 0; for (int i = 0; i < n; ++i) S += a[i];
    ll cur = 0, mx = 0;
    for (int i = 0; i < n; ++i) {
        cur += a[i];
        if (i >= x) cur -= a[i - x];
        mx = max(mx, cur);
    }
    return min(S / 2, S - mx);
}
int main() {
    freopen("pairing.in", "r", stdin);
    freopen("pairing.out", "w", stdout);
    ios::sync_with_stdio(0); cin.tie(0);
    cin >> n >> y;
```

余博士教编程

```cpp
    for (int i = 0; i < n; ++i) cin >> a[i];
    if (dishes(1) < y) { cout << -1 << '\n'; return 0; }
    ll L = 1, R = n, ans = 1;
    while (L <= R) {
        ll mid = (L + R) >> 1;
        if (dishes(mid) >= y) { ans = mid; L = mid + 1; }
        else R = mid - 1;
    }
    cout << ans << '\n';
    return 0;
}
```

逃离后室（liminal）

本题正解所涉及的知识点主要包括**广度优先搜索（0-1 BFS）、图论建模、数论（gcd）**。其中“广度优先搜索”属于“图论算法”，大纲标注学习难度系数为 4；“0-1 BFS（双端队列 BFS）”属于“图论算法的常用优化”，大纲标注学习难度系数为 4；“gcd”属于“初等数论”，大纲标注学习难度系数为 2。本题所涉知识点均在 NOI 大纲所规定的 CSP-S（提高级）难度系数范围内。

测试点 1~2：$n, m \le 20,\ s < 50$

思路

初始 SAN 值 $s < 50$ 很小，把“当前剩余 SAN 值 r”作为状态一维做普通 BFS，状态为 (l, x, y, r)。

- 移动：$r \to r-1$（消耗 1 点 SAN），需 $r - 1 \ge 0$；
- 卡层：r 不变，目标层 l' 满足 $l' \equiv r \cdot t \pmod{k}$，即 l' 是 $\gcd(r, k)$ 的倍数。

第一次搜到 `@` 时记下最大剩余 SAN。由于 $s < 50$，状态数很小。

复杂度

状态数 $O(k \cdot n \cdot m \cdot s)$，$s < 50$ 时很小。

```cpp
#include <bits/stdc++.h>
using namespace std;
int n, m, k; char g[12][25][25];
struct S { int l, x, y, r; };
int gcd_(int a, int b) { return b ? gcd_(b, a % b) : a; }
int main() {
    int T; cin >> T;
    while (T--) {
        long long s; cin >> n >> m >> k >> s;
        int sl, sx, sy;
        for (int l = 0; l < k; ++l)
            for (int i = 0; i < n; ++i)
                for (int j = 0; j < m; ++j) {
                    cin >> g[l][i][j];
                    if (g[l][i][j] == 'M') { sl = l; sx = i; sy = j; }
                }
        map<pair<tuple<int,int,int>,int>,int> dist;
        queue<S> q; S st{sl, sx, sy, (int)s};
```

余博士教编程

```
        dist[make_pair(make_tuple(sl, sx, sy), (int)s)] = 0;
        q.push(st); int best = -1;
        while (!q.empty()) {
            S u = q.front(); q.pop();
            int d = dist[make_pair(make_tuple(u.l, u.x, u.y), u.r)];
            if (g[u.l][u.x][u.y] == '@') { best = max(best, u.r); continue;
}
            int dr[4][2] = {{1,0},{-1,0},{0,1},{0,-1}};
            for (int t = 0; t < 4; ++t) {
                int nx = u.x + dr[t][0], ny = u.y + dr[t][1];
                if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
                if (g[u.l][nx][ny] == '!' || u.r - 1 < 0) continue;
                auto key = make_pair(make_tuple(u.l, nx, ny), u.r - 1);
                if (!dist.count(key)) { dist[key] = d + 1; q.push({u.l, nx,
ny, u.r - 1}); }
            }
            int gg = gcd_(u.r, k);
            for (int l2 = 0; l2 < k; ++l2)
                if (l2 % gg == 0 && g[l2][u.x][u.y] != '!') {
                    auto key = make_pair(make_tuple(l2, u.x, u.y), u.r);
                    if (!dist.count(key)) { dist[key] = d; q.push({l2, u.x,
u.y, u.r}); }
                }
        }
        if (best == -1) cout << "Maybe I'm lost.\n";
        else cout << best << '\n';
    }
    return 0;
}
```

测试点 3~5：$k = 1$

思路

只有一层，卡层没有意义，退化成在一张图上从 `M` 走到 `@` 的最少步数，普通 BFS 即可。答案 = 初始 SAN − 最短步数（不可达或步数 $> s$ 则输出 `Maybe I'm lost.`）。

复杂度

$O(nm)$。

```
#include <bits/stdc++.h>
using namespace std;
int n, m, k; char g[505][505]; int d[505][505];
int main() {
    int T; cin >> T;
    while (T--) {
        long long s; cin >> n >> m >> k >> s;
        int sx, sy;
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < m; ++j) {
                cin >> g[i][j];
                if (g[i][j] == 'M') { sx = i; sy = j; }
```

```
            }
        memset(d, -1, sizeof d);
        queue<pair<int,int>> q; d[sx][sy] = 0; q.push({sx, sy});
        int ans = -1;
        while (!q.empty()) {
            int x = q.front().first, y = q.front().second; q.pop();
            if (g[x][y] == '@') { ans = d[x][y]; break; }
            int dr[4][2] = {{1,0},{-1,0},{0,1},{0,-1}};
            for (int t = 0; t < 4; ++t) {
                int nx = x + dr[t][0], ny = y + dr[t][1];
                if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
                if (g[nx][ny] == '!' || d[nx][ny] != -1) continue;
                d[nx][ny] = d[x][y] + 1; q.push({nx, ny});
            }
        }
        if (ans == -1 || ans > s) cout << "Maybe I'm lost.\n";
        else cout << s - ans << '\n';
    }
    return 0;
}
```

测试点 6~13：$k \le 3$ / $k \le 10$

思路

卡层能到达的层集合只与 $\gcd(r, k)$ 有关，而 r 在移动过程中模 k 的余数决定 $\gcd(r, k)$。因此把“剩余 SAN 模 k 的余数”$\mathrm{rmod} = r \bmod k$ 加入状态，状态为 (l, x, y, rmod)：

- 移动一步：$\mathrm{rmod} \to (\mathrm{rmod} - 1) \bmod k$，边权 1；
- 卡层：rmod 不变，可到达所有 $\gcd(\mathrm{rmod}, k)$ 的倍数层，边权 0。

既有边权 0 又有边权 1，用**双端队列 BFS（0-1 BFS）**：边权 1 入队尾，边权 0 入队首。

复杂度

状态数 $O(k^2nm)$，0-1 BFS 每个状态入队出队一次，总复杂度 $O(k^2nm)$。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
int n, m, k; ll s;
char g[10][505][505];
int dist[25000005];
bool vis[505][505][10];
vector<int> tele[10];
int main() {
    freopen("liminal.in", "r", stdin);
    freopen("liminal.out", "w", stdout);
    ios::sync_with_stdio(0); cin.tie(0);
    int T; cin >> T;
    while (T--) {
        cin >> n >> m >> k >> s;
        int sl = -1, sx = -1, sy = -1;
        for (int l = 0; l < k; ++l)
```

```
            for (int i = 0; i < n; ++i) {
                cin >> g[l][i];
                for (int j = 0; j < m; ++j)
                    if (g[l][i][j] == 'M') { sl = l; sx = i; sy = j; }
            }
        for (int r = 0; r < k; ++r) {
            tele[r].clear();
            int gg = std::__gcd(r, k);
            for (int l = 0; l < k; ++l) if (l % gg == 0)
tele[r].push_back(l);
        }
        int total = k * n * m * k;
        fill(dist, dist + total, -1);
        memset(vis, 0, sizeof(vis));
        auto idx = [&](int l, int x, int y, int r) {
            return ((l * n + x) * m + y) * k + r;
        };
        int r0 = (int)(s % k);
        int st = idx(sl, sx, sy, r0);
        dist[st] = 0; deque<int> dq; dq.push_back(st);
        int best = -1;
        while (!dq.empty()) {
            int u = dq.front(); dq.pop_front();
            int d = dist[u];
            if ((ll)d > s) continue;
            int r = u % k, t = u / k;
            int y = t % m; t /= m;
            int x = t % n; int l = t / n;
            if (g[l][x][y] == '@') { best = d; break; }
            int nr = (r + k - 1) % k;
            int dr[4][2] = {{1,0},{-1,0},{0,1},{0,-1}};
            for (int t2 = 0; t2 < 4; ++t2) {
                int nx = x + dr[t2][0], ny = y + dr[t2][1];
                if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
                if (g[l][nx][ny] == '!') continue;
                int v = idx(l, nx, ny, nr);
                if (dist[v] == -1) { dist[v] = d + 1; dq.push_back(v); }
            }
            if (!vis[x][y][r]) {
                for (int l2 : tele[r]) {
                    if (l2 == l) continue;
                    if (g[l2][x][y] == '!') continue;
                    int v = idx(l2, x, y, r);
                    if (dist[v] == -1) { dist[v] = d; dq.push_front(v); }
                }
                vis[x][y][r] = 1;
            }
        }
        if (best == -1) cout << "Maybe I'm lost.\n";
        else cout << s - best << '\n';
    }
    return 0;
}
```

步线行针（lattice）

本题正解所涉及的知识点主要包括**组合数学、递推、矩阵快速幂、滚动数组**。其中“组合数学”属于“初等数学/组合”，大纲标注学习难度系数为 4；“递推”属于“基础算法”，大纲标注学习难度系数为 3；“矩阵快速幂”属于“高级算法”，大纲标注学习难度系数为 6；“滚动数组”属于“空间优化技巧”，大纲标注学习难度系数为 2。本题所涉知识点在 NOI 大纲所规定的 CSP-S（提高级）难度系数范围内。

测试点 1：$k = 1$

思路

直线 $y = -x + d$ 上的非负整数点是 $(i,\ d - i)$，权值为 $C_{i+d-i}^{i} = C_{d}^{i}$。故答案为 $\sum_{i=0}^{d} C_{d}^{i} = 2^{d} \pmod{998244353}$。用快速幂即可。

复杂度

$O(\log d)$。

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll MOD = 998244353;
ll qpow(ll a, ll b) {
    ll r = 1;
    while (b) { if (b & 1) r = r * a % MOD; a = a * a % MOD; b >>= 1; }
    return r;
}
int main() {
    ll k, d; cin >> k >> d;
    cout << qpow(2, d) << '\n';
    return 0;
}
```

测试点 2~4：$d \le 10^6$（或 $k \times d \le 10^6$）

思路

直线 $y = -kx + d$ 上的非负整数点为 $(i,\ d - ki)$，$i = 0..\lfloor d/k \rfloor$。点 (i, j) 的权值为 C_{i+j}^{i}，代入 $j = d - ki$ 得 $i + j = d - (k-1)i$。故答案为

$$\sum_{i=0}^{\lfloor d/k \rfloor} C_{d-(k-1)i}^{i} \pmod{998244353}$$

预处理阶乘与逆元后直接求和。

复杂度

预处理 $O(\max d)$，求和 $O(d/k)$。

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
```

```cpp
const ll MOD = 998244353;
ll fac[1000005], invf[1000005];
ll qpow(ll a, ll b) { ll r = 1; while (b) { if (b & 1) r = r * a % MOD; a = a * a % MOD; b >>= 1; } return r; }
ll C(int n, int m) { if (n < m || m < 0) return 0; return fac[n] * invf[m] % MOD * invf[n - m] % MOD; }
int main() {
    ll k, d; cin >> k >> d;
    int L = (int)(d + 5);
    fac[0] = invf[0] = 1;
    for (int i = 1; i <= L; ++i) fac[i] = fac[i - 1] * i % MOD;
    invf[L] = qpow(fac[L], MOD - 2);
    for (int i = L; i >= 1; --i) invf[i - 1] = invf[i] * i % MOD;
    ll ans = 0;
    for (int i = 0; (ll)i * k <= d; ++i)
        ans = (ans + C((int)(d - (k - 1) * i), i)) % MOD;
    cout << ans << '\n';
    return 0;
}
```

测试点 5~9：$k \le 100$（矩阵快速幂）

思路

点 (i,j) 的权值 $f(i,j)$ 满足 $f(i,j)=f(i-1,j)+f(i,j-1)$。把直线上所有点权值之和记成 $g(k,d)$，对每个点套用上式再求和可得递推

$$g(k,d)=g(k,d-1)+g(k,d-k)$$

边界：$d<k$ 时直线上只有 $(0,d)$ 一点，权值为 1，故 $g(k,0)=\cdots=g(k,k-1)=1$。这是 k 阶线性递推，用矩阵快速幂 $O(k^3\log d)$ 求解。

复杂度

$O(k^3\log d)$。

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N = 110;
const ll MOD = 998244353;
struct Matrix {
    int n, m; ll mx[N][N];
    Matrix() : n(1), m(1) { memset(mx, 0, sizeof(mx)); }
    Matrix(int n, int m) : n(n), m(m) { memset(mx, 0, sizeof(mx)); }
    Matrix operator * (const Matrix& t) const {
        Matrix ans(n, t.m);
        for (int i = 1; i <= n; ++i)
            for (int o = 1; o <= m; ++o) if (mx[i][o])
                for (int j = 1; j <= t.m; ++j)
                    ans.mx[i][j] = (ans.mx[i][j] + mx[i][o] * t.mx[o][j]) % MOD;
        return ans;
    }
```

```
};
Matrix qsm(Matrix a, ll b) {
    Matrix res(a.n, a.n);
    for (int i = 1; i <= a.n; ++i) res.mx[i][i] = 1;
    while (b) { if (b & 1) res = res * a; a = a * a; b >>= 1; }
    return res;
}
int main() {
    ll k, d; cin >> k >> d;
    if (d < k) { cout << 1 << '\n'; return 0; }
    Matrix F0(1, k), T(k, k);
    for (int i = 1; i <= k; ++i) F0.mx[1][i] = 1;
    T.mx[1][k]++; T.mx[k][k]++;
    for (int i = 2; i <= k; ++i) T.mx[i][i - 1] = 1;
    cout << (F0 * qsm(T, d - k + 1)).mx[1][k] << '\n';
    return 0;
}
```

测试点 10~14：$d \le 3 \times 10^7$（滚动数组）

思路

d 最多 3×10^7，直接递推，但不必开 $O(d)$ 大数组——递推 $g(n) = g(n-1) + g(n-k)$ 只需最近 k 个值，用长度为 k 的循环数组滚动维护，时间 $O(d)$，空间 $O(k)$。

复杂度

时间 $O(d)$，空间 $O(k)$。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll MOD = 998244353;
const int M = 100010;
ll a[M];
int main() {
    ll k, d; cin >> k >> d;
    for (int i = 0; i < k; ++i) a[i] = 1;
    int now = 0, pre = k - 1;
    for (int i = k; i <= d; ++i) {
        a[now] = (a[pre] + a[now]) % MOD;
        pre = now++;
        if (now >= k) now = 0;
    }
    cout << a[d % k] << '\n';
    return 0;
}
```

测试点 15~20：$k \times d \le 10^{10}$（正解）

思路

数据满足 $k \times d \le 10^{10}$，于是 $\min(k, d) \le 10^5$。把前面两档结合分类讨论：

- 若 $k \le 100$：矩阵快速幂 $O(k^3 \log d)$；
- 否则 $k > 100$：由 $k \times d \le 10^{10}$ 得 $d < 10^8$ 且 $k \le 10^5$，用循环数组递推 $O(d)$。

注意 $k = 1$ 时递推退化为 $g(n) = 2g(n-1)$，走矩阵分支时 T 变成 1×1 的 $[2]$，结果仍是 2^d。

复杂度

$k \le 100$ 时 $O(k^3 \log d)$；$k > 100$ 时 $O(d)$。

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N = 110, M = 100010;
const ll mod = 998244353;
struct Matrix {
    int n, m; ll mx[N][N];
    Matrix() : n(1), m(1) { memset(mx, 0, sizeof(mx)); }
    Matrix(int n, int m) : n(n), m(m) { memset(mx, 0, sizeof(mx)); }
    Matrix operator * (const Matrix& t) const {
        Matrix ans(n, t.m);
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= t.m; j++)
                for (int o = 1; o <= m; o++)
                    ans.mx[i][j] = (ans.mx[i][j] + mx[i][o] * t.mx[o][j] %
mod) % mod;
        return ans;
    }
};
Matrix F0, T; ll k, d, a[M];
Matrix qsm(Matrix a, ll b) {
    Matrix res(a.n, a.n);
    for (int i = 1; i <= res.n; i++) res.mx[i][i] = 1;
    while (b) { if (b & 1) res = res * a; a = a * a; b >>= 1; }
    return res;
}
int main() {
    freopen("lattice.in", "r", stdin);
    freopen("lattice.out", "w", stdout);
    ios::sync_with_stdio(0); cin.tie(0);
    cin >> k >> d;
    if (d < k) {
        cout << 1 << '\n';
    } else if (k <= 100) {
        T.n = T.m = F0.m = k; F0.n = 1;
        for (int i = 1; i <= k; i++) F0.mx[1][i] = 1;
        T.mx[1][k]++, T.mx[k][k]++;
        for (int i = 2; i <= k; i++) T.mx[i][i - 1]++;
        cout << (F0 * qsm(T, d - k + 1)).mx[1][k] << '\n';
    } else {
        for (int i = 0; i < k; i++) a[i] = 1;
        int now = 0, pre = k - 1;
```

```
        for (int i = k; i <= d; i++) {
            a[now] = (a[pre] + a[now]) % mod;
            pre = now++;
            if (now >= k) now = 0;
        }
        cout << a[d % k] << '\n';
    }
    return 0;
}
```

巨兽追猎（hunt）

本题正解所涉及的知识点主要包括**排序、二分、前缀/后缀最值、单调栈/单调队列、线段树（或 CDQ/分治）**。其中“排序”与“二分”属于“基础算法”，大纲标注学习难度系数分别为 3 和 4；“前缀/后缀最值”属于“算法策略”，大纲标注学习难度系数为 3；“线段树/分治”属于“高级算法”，大纲标注学习难度系数为 6。本题所涉知识点在 NOI 大纲所规定的 CSP-S（提高级）难度系数范围内。

核心转化

第 j 把武器在第 i 阶段的伤害为 $\min(A_jV_i,\ P_j)$，所以第 i 阶段的最高伤害

$M_i = \max_j \min(A_jV_i,\ P_j)$

问题化为：求 $\sum_{1\le l\le r\le n} \max(0,\ \max_{i\in[l,r]} M_i - \min_{i\in[l,r]} B_i)$。

测试点 1~2：$n, k \le 2000$

思路

直接 $O(nk)$ 求出每个阶段的最高伤害 M_i，再 $O(n^2)$ 枚举所有区间，边枚举边维护 $\max M$ 与 $\min B$。

复杂度

$O(n^2 + nk)$。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll MOD = 998244353;
int n, k;
ll V[2005], B[2005], A[2005], P[2005], M[2005];
int main() {
    cin >> n >> k;
    for (int i = 0; i < n; ++i) cin >> V[i];
    for (int i = 0; i < n; ++i) cin >> B[i];
    for (int i = 0; i < k; ++i) cin >> A[i] >> P[i];
    for (int i = 0; i < n; ++i) {
        M[i] = 0;
        for (int j = 0; j < k; ++j) M[i] = max(M[i], min(A[j] * V[i],
P[j]));
```

```
    }
    ll ans = 0;
    for (int l = 0; l < n; ++l) {
        ll mx = 0, mn = 1e18;
        for (int r = l; r < n; ++r) {
            mx = max(mx, M[r]); mn = min(mn, B[r]);
            ans = (ans + max(0LL, mx - mn)) % MOD;
        }
    }
    cout << ans << '\n';
    return 0;
}
```

测试点 3~4：只有一把武器（$k = 1$）

思路

$k = 1$ 时 $M_i = \min(A_1 V_i,\ P_1)$，$O(n)$ 直接算出 M。区间贡献部分与后面的通用方法相同：用「单调栈 + 双指针」或「线段树」求 $\sum \max(0, \max M - \min B)$。下面 5_{6}、$_{7}8$ 分别给出利用特殊性质的更简做法；本题档可直接复用 9~14 的线段树方法 $O(n \log n)$。

复杂度

求 M 为 $O(n)$；区间贡献 $O(n \log n)$。

测试点 5~6：所有 V_i 相等

思路

所有 V_i 相同，则每个阶段的 M_i 也相同，记为常数 M_0。此时区间价值为 $\max(0,\ M_0 - \min B)$。对每个位置 i，用单调栈求出 B_i 作为区间最小值的左右边界 $L[i], R[i]$，则 B_i 是最小值的子区间个数为 $(i - L[i])(R[i] - i)$，贡献为 $\max(0, M_0 - B_i) \times (i - L[i])(R[i] - i)$。

复杂度

$O(n)$。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll mod = 998244353;
int main() {
    ios::sync_with_stdio(0); cin.tie(0);
    int n, k; cin >> n >> k;
    vector<ll> V(n + 1), B(n + 1);
    for (int i = 1; i <= n; i++) cin >> V[i];
    for (int i = 1; i <= n; i++) cin >> B[i];
    vector<ll> A(k), P(k);
    for (int j = 0; j < k; j++) cin >> A[j] >> P[j];
    ll M0 = 0;
    for (int j = 0; j < k; j++) M0 = max(M0, min(A[j] * V[1], P[j]));
    vector<int> L(n + 1), R(n + 1), st;
```

```cpp
    for (int i = 1; i <= n; i++) {
        while (!st.empty() && B[st.back()] >= B[i]) st.pop_back();
        L[i] = st.empty() ? 0 : st.back();
        st.push_back(i);
    }
    st.clear();
    for (int i = n; i >= 1; i--) {
        while (!st.empty() && B[st.back()] > B[i]) st.pop_back();
        R[i] = st.empty() ? n + 1 : st.back();
        st.push_back(i);
    }
    ll ans = 0;
    for (int i = 1; i <= n; i++) {
        ll d = max(0LL, M0 - B[i]);
        ll cnt = (ll)(i - L[i]) * (R[i] - i) % mod;
        ans = (ans + d % mod * cnt) % mod;
    }
    cout << ans << '\n';
    return 0;
}
```

测试点 7~8：B_i 单调不增

思路

B 单调不增，故 $\min_{i\in[l,r]} B_i = B_r$（最小值在右端点）。答案化为

$$\sum_{r=1}^{n}\sum_{l=1}^{r}\max\left(0,\ \max_{i\in[l,r]} M_i - B_r\right)$$

对每个 r，用**递减单调栈**维护 M 的"左端到 r 的区间最大值"分段（从左到右 M 递减），并维护分段长度与 $\sum M\times$ 长度 的前缀和；再二分找到最后一个 $M > B_r$ 的分段，用前缀和 $O(1)$ 累计贡献。

复杂度

$O(n\log n)$（栈上二分），空间 $O(n)$。

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll mod = 998244353;
int main() {
    ios::sync_with_stdio(0); cin.tie(0);
    int n, k; cin >> n >> k;
    vector<ll> V(n + 1), B(n + 1);
    for (int i = 1; i <= n; i++) cin >> V[i];
    for (int i = 1; i <= n; i++) cin >> B[i];
    vector<ll> A(k), P(k);
    for (int j = 0; j < k; j++) cin >> A[j] >> P[j];
    vector<ll> M(n + 1);
    for (int i = 1; i <= n; i++) {
        ll best = 0;
        for (int j = 0; j < k; j++) best = max(best, min(A[j] * V[i],
P[j]));
        M[i] = best;
```

```
    }
    vector<ll> val, len, sumV, sumL;
    sumV.push_back(0); sumL.push_back(0);
    ll ans = 0;
    for (int r = 1; r <= n; r++) {
        ll w = 1;
        while (!val.empty() && val.back() <= M[r]) {
            w += len.back();
            val.pop_back(); len.pop_back();
            sumV.pop_back(); sumL.pop_back();
        }
        val.push_back(M[r]); len.push_back(w);
        sumV.push_back(sumV.back() + M[r] % mod * (w % mod) % mod);
        sumL.push_back(sumL.back() + w % mod);
        int lo = 0, hi = (int)val.size();
        while (lo < hi) {
            int mid = (lo + hi) >> 1;
            if (val[mid] > B[r]) lo = mid + 1; else hi = mid;
        }
        int pos = lo;
        if (pos > 0) {
            ll s = sumV[pos] % mod, c = sumL[pos] % mod;
            ans = (ans + s - B[r] % mod * c % mod + mod) % mod;
        }
    }
    cout << ans << '\n';
    return 0;
}
```

测试点 9~14：线段树 + 单调栈（$n, k \le 2 \times 10^5$）

思路

枚举右端点 r，对每个左端点 l 维护 $D[l] = \max_{i\in[l,r]} M_i - \min_{i\in[l,r]} B_i$。固定 r 时 $D[l]$ 关于 l 单调不增，所以正的 $D[l]$ 是 l 的一个前缀；随 r 右移 $D[l]$ 单调不减。

- 用两个单调栈分别维护「窗口 $\max M$ 的分段」和「$\min B$ 的分段」，新元素压入时对受影响段做区间加减；
- 用一棵支持「区间加 + 区间和 + 查询最后一个正数」的线段树维护 $D[l]$；
- 每次把正前缀和累加进答案。

复杂度

$O(n \log n)$，空间 $O(n)$。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;
const ll MOD = 998244353;
vector<ll> mn, mx, lazy, sum;
inline ll modNorm(ll x) { x %= MOD; if (x < 0) x += MOD; return x; }
inline void apply(int o, int l, int r, ll value) {
```

```
    mn[o] += value; mx[o] += value; lazy[o] += value;
    ll add = modNorm(value) * (r - l + 1LL) % MOD;
    sum[o] += add; if (sum[o] >= MOD) sum[o] -= MOD;
}
inline void push(int o, int l, int r) {
    if (lazy[o] == 0 || l == r) return;
    int mid = (l + r) >> 1;
    apply(o << 1, l, mid, lazy[o]);
    apply(o << 1 | 1, mid + 1, r, lazy[o]);
    lazy[o] = 0;
}
inline void pull(int o) {
    mn[o] = min(mn[o << 1], mn[o << 1 | 1]);
    mx[o] = max(mx[o << 1], mx[o << 1 | 1]);
    sum[o] = sum[o << 1] + sum[o << 1 | 1];
    if (sum[o] >= MOD) sum[o] -= MOD;
}
void setPoint(int o, int l, int r, int pos, ll value) {
    if (l == r) { mn[o] = mx[o] = value; lazy[o] = 0; sum[o] =
modNorm(value); return; }
    push(o, l, r);
    int mid = (l + r) >> 1;
    if (pos <= mid) setPoint(o << 1, l, mid, pos, value);
    else setPoint(o << 1 | 1, mid + 1, r, pos, value);
    pull(o);
}
void addRange(int o, int l, int r, int L, int R, ll value) {
    if (L <= l && r <= R) { apply(o, l, r, value); return; }
    push(o, l, r);
    int mid = (l + r) >> 1;
    if (L <= mid) addRange(o << 1, l, mid, L, R, value);
    if (R > mid) addRange(o << 1 | 1, mid + 1, r, L, R, value);
    pull(o);
}
int lastPositive(int o, int l, int r, int right) {
    if (l > right) return 0;
    if (r <= right && mx[o] <= 0) return 0;
    if (r <= right && mn[o] > 0) return r;
    if (l == r) return l;
    push(o, l, r);
    int mid = (l + r) >> 1;
    int res = lastPositive(o << 1 | 1, mid + 1, r, right);
    if (res) return res;
    return lastPositive(o << 1, l, mid, right);
}
ll querySum(int o, int l, int r, int L, int R) {
    if (L <= l && r <= R) return sum[o];
    push(o, l, r);
    int mid = (l + r) >> 1; ll res = 0;
    if (L <= mid) res += querySum(o << 1, l, mid, L, R);
    if (R > mid) res += querySum(o << 1 | 1, mid + 1, r, L, R);
    if (res >= MOD) res -= MOD;
    return res;
}
struct Part { int left, right; ll value; };
```

```cpp
int main() {
    ios::sync_with_stdio(0); cin.tie(0), cout.tie(0);
    int n, k; cin >> n >> k;
    vector<ll> V(n + 1), B(n + 1);
    for (int i = 1; i <= n; i++) cin >> V[i];
    for (int i = 1; i <= n; i++) cin >> B[i];
    vector<pair<ll, ll>> weapons(k);
    for (int i = 0; i < k; i++) cin >> weapons[i].first >>
weapons[i].second;
    sort(weapons.begin(), weapons.end(), [](const pair<ll,ll>& x, const
pair<ll,ll>& y) {
        return (i128)x.second * y.first < (i128)y.second * x.first;
    });
    vector<ll> suffixMaxA(k + 1, 0);
    for (int i = k - 1; i >= 0; i--) suffixMaxA[i] = max(suffixMaxA[i + 1],
weapons[i].first);
    vector<int> order(n); iota(order.begin(), order.end(), 1);
    sort(order.begin(), order.end(), [&](int x, int y) {
        if (V[x] != V[y]) return V[x] < V[y];
        return x < y;
    });
    vector<ll> M(n + 1); ll bestCappedP = 0; int ptr = 0;
    for (int id : order) {
        ll v = V[id];
        while (ptr < k && (i128)weapons[ptr].second <
(i128)weapons[ptr].first * v) {
            bestCappedP = max(bestCappedP, weapons[ptr].second); ptr++;
        }
        ll best = bestCappedP;
        if (ptr < k) best = max(best, v * suffixMaxA[ptr]);
        M[id] = best;
    }
    mn.assign(4 * n + 5, 0); mx.assign(4 * n + 5, 0);
    lazy.assign(4 * n + 5, 0); sum.assign(4 * n + 5, 0);
    vector<Part> maxStack, minStack;
    ll answer = 0;
    for (int r = 1; r <= n; r++) {
        setPoint(1, 1, n, r, M[r] - B[r]);
        int left = r;
        while (!maxStack.empty() && maxStack.back().value <= M[r]) {
            Part cur = maxStack.back(); maxStack.pop_back();
            addRange(1, 1, n, cur.left, cur.right, M[r] - cur.value);
            left = cur.left;
        }
        maxStack.push_back({left, r, M[r]});
        left = r;
        while (!minStack.empty() && minStack.back().value >= B[r]) {
            Part cur = minStack.back(); minStack.pop_back();
            addRange(1, 1, n, cur.left, cur.right, cur.value - B[r]);
            left = cur.left;
        }
        minStack.push_back({left, r, B[r]});
        int last = lastPositive(1, 1, n, r);
        if (last > 0) {
            answer += querySum(1, 1, n, 1, last);
```

```
            if (answer >= MOD) answer -= MOD;
        }
    }
    cout << answer << '\n';
    return 0;
}
```

测试点 15~20：$n, k \le 5 \times 10^5$（正解）

思路

分三步。

第一步：求 M_i。 把武器按 P_j/A_j 排序，维护前缀最大 P、后缀最大 A；对每个 V_i 二分出「封顶/未封顶」分界，$M_i = \max(V_i \cdot \text{后缀最大}A,\ \text{前缀最大}P)$。用 `__int128` 比较避免乘法溢出。

第二步：求 $z[l]$。 令 $z[l]$ 为最小的 r 使 $\max_{i\in[l,r]} M_i > \min_{i\in[l,r]} B_i$（不存在记为 $n+1$）。用双指针 + 两个单调队列在 $O(n)$ 内求出所有 $z[l]$，区间 $[l, r]$ 有贡献当且仅当 $r \ge z[l]$。

第三步：统计。 答案 $= \sum$ 合法区间的 $\max M - \sum$ 合法区间的 $\min B$。对 M、B 分别用单调栈求每个位置作为最值的左右边界，再结合 $z[l]$ 单调不减的性质，用二分 + 前缀和数出合法区间个数，乘上 M_i（或 B_i）累加，最后相减取模。

复杂度

求 M 为 $O((n+k)\log k)$，求 z 为 $O(n)$，统计为 $O(n \log n)$；空间 $O(n)$。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll MOD = 998244353;

int main() {
    freopen("hunt.in", "r", stdin);
    freopen("hunt.out", "w", stdout);
    int n, k;
    scanf("%d %d", &n, &k);
    vector<ll> V(n + 1), B(n + 1);
    for (int i = 1; i <= n; i++) scanf("%lld", &V[i]);
    for (int i = 1; i <= n; i++) scanf("%lld", &B[i]);
    vector<pair<ll, ll>> w(k);
    for (int i = 0; i < k; i++) scanf("%lld %lld", &w[i].first,
&w[i].second);

    sort(w.begin(), w.end(), [](const pair<ll,ll>& a, const pair<ll,ll>& b)
{
        return (__int128)a.second * b.first < (__int128)b.second * a.first;
    });
    vector<ll> prefP(k), sufA(k);
    prefP[0] = w[0].second;
    for (int i = 1; i < k; i++) prefP[i] = max(prefP[i-1], w[i].second);
    sufA[k-1] = w[k-1].first;
    for (int i = k-2; i >= 0; i--) sufA[i] = max(sufA[i+1], w[i].first);
    vector<ll> M(n + 1);
```

```
    for (int i = 1; i <= n; i++) {
        ll v = V[i];
        int lo = 0, hi = k, pos = k;
        while (lo < hi) {
            int mid = (lo + hi) / 2;
            if ((__int128)w[mid].first * v <= w[mid].second) hi = mid;
            else lo = mid + 1;
        }
        pos = lo;
        ll bestCapped = (pos > 0 ? prefP[pos-1] : 0);
        ll bestUncapped = (pos < k ? v * sufA[pos] : 0);
        M[i] = max(bestCapped, bestUncapped);
    }

    const int INF = n + 1;
    vector<int> z(n + 2, INF);
    {
        int r = 0;
        deque<int> mx, mn;
        for (int l = 1; l <= n; l++) {
            while (!mx.empty() && mx.front() < l) mx.pop_front();
            while (!mn.empty() && mn.front() < l) mn.pop_front();
            while (r < n) {
                if (r >= l && M[mx.front()] > B[mn.front()]) break;
                int q = ++r;
                while (!mx.empty() && M[mx.back()] <= M[q]) mx.pop_back();
                mx.push_back(q);
                while (!mn.empty() && B[mn.back()] >= B[q]) mn.pop_back();
                mn.push_back(q);
            }
            z[l] = (r >= l && M[mx.front()] > B[mn.front()]) ? r : INF;
        }
    }
    vector<ll> preZ(n + 1, 0);
    for (int l = 1; l <= n; l++) preZ[l] = preZ[l - 1] + z[l];

    auto solve = [&](const vector<ll>& val, bool asMax) -> ll {
        vector<int> L(n + 1, 0), R(n + 1, n + 1), st;
        for (int i = 1; i <= n; i++) {
            while (!st.empty() && (asMax ? val[st.back()] < val[i] :
val[st.back()] > val[i])) st.pop_back();
            L[i] = st.empty() ? 0 : st.back();
            st.push_back(i);
        }
        st.clear();
        for (int i = n; i >= 1; i--) {
            while (!st.empty() && (asMax ? val[st.back()] <= val[i] :
val[st.back()] >= val[i])) st.pop_back();
            R[i] = st.empty() ? n + 1 : st.back();
            st.push_back(i);
        }
        ll total = 0;
        for (int i = 1; i <= n; i++) {
            int a = L[i] + 1, b = i, lim = R[i] - 1;
            if (a > b || z[a] > lim) continue;
```

```
            int lo = a, hi = b, ls = a - 1;
            while (lo <= hi) {
                int mid = (lo + hi) >> 1;
                if (z[mid] <= lim) { ls = mid; lo = mid + 1; }
                else hi = mid - 1;
            }
            if (ls < a) continue;
            int lo2 = a, hi2 = ls, p = a - 1;
            while (lo2 <= hi2) {
                int mid = (lo2 + hi2) >> 1;
                if (z[mid] <= i) { p = mid; lo2 = mid + 1; }
                else hi2 = mid - 1;
            }
            ll cnt = 0;
            if (p >= a) cnt += (ll)(p - a + 1) * (lim - i + 1);
            int q0 = max(p + 1, a);
            if (q0 <= ls) cnt += (ll)(ls - q0 + 1) * (ll)(lim + 1) -
(preZ[ls] - preZ[q0 - 1]);
            total = (total + (val[i] % MOD) * (cnt % MOD)) % MOD;
        }
        return total;
    };

    ll sm = solve(M, true), sn = solve(B, false);
    printf("%lld\n", ((sm - sn) % MOD + MOD) % MOD);
    return 0;
}
```

余博士教编程